

Accelerating Complex Event Recognition via Range Bitmap-based Indexes with Window-wise Filtering

Shizhe Liu, Haipeng Dai, *Senior Member, IEEE*, Shaoxu Song, *Senior Member, IEEE*, Meng Li, Yuemeng Zhang, Hancheng Wang, Du Zhang, Rong Gu, *Member, IEEE*, Guihai Chen, *Fellow, IEEE*

Abstract—Complex event recognition (CER) refers to identifying specific patterns composed of several primitive events in event stores. Since full-scanning event stores to identify primitive events that hold query constraint conditions incurs costly I/O overhead, a mainstream and practical approach is to use index techniques to obtain these events. However, prior index-based approaches suffer from significant I/O and sorting overhead when processing the query with high predicate selectivity or long query window, which leads to high query latency. To address this issue, we propose ACER, a Range Bitmap-based index, to accelerate CER. Firstly, ACER achieves a low index space overhead by grouping the events with the same type into a cluster and compressing the cluster data, reducing I/O overhead when reading indexes. Secondly, ACER builds Range Bitmaps for queried attributes and ensures that the events of each cluster in the index block are chronologically ordered. Then, ACER can always obtain ordered query results for a specific event type through merge operations, avoiding sorting overhead. Most importantly, ACER avoids unnecessary disk accesses in indexes and events via window-wise filtering, thus reducing the I/O overhead further. Lastly, we propose an enhanced version of ACER (ACER-E) by optimizing the read/write operation of index blocks and variable query order. Our extensive experiments demonstrate that ACER and ACER-E reduce the query latency by up to one order of magnitude compared with SOTA techniques.

Index Terms—Complex event recognition, bitmap indexes, window-wise filtering.

I. INTRODUCTION

COMPLEX event recognition (CER) refers to identifying specific patterns composed of several primitive events in event stores. Here, the primitive events are simple, derived and timestamped [1], and event stores preserve a long sequence of these events. Currently, CER is widely employed in pattern mining applications like cluster monitoring [2]–[4], network intrusion detection [5]–[7], and financial services [8]–[10]. Notably, in 2016, the International Organization for Standardization issued new semantics in SQL to support CER [11]. Meanwhile, prominent database and data-stream systems, including Oracle, Snowflake, and Flink have already implemented CER interfaces [12]–[14].

To better understand CER, we give the following example.

Example. In the stock trade market, an investor identifies an unusual trading pattern involving two positively correlated stocks, GOOG and MSFT, as shown in Table I. This pattern is characterized by a first increase in MSFT's opening price

Shizhe Liu, Haipeng Dai, Meng Li, Yuemeng Zhang, Hancheng Wang, Rong Gu, and Guihai Chen are with the State Key Laboratory for Novel Software Technology, Nanjing University. E-mail: {shizhelu, yuemengzhang, hanchengwang}@mail.nju.edu.cn, {haipengdai, meng, gurong, gchen}@nju.edu.cn. Haipeng Dai is the corresponding author.

Shaoxu Song is with Tsinghua University. E-mail: sxsong@tsinghua.edu.cn.

Du Zhang is with Nankai University. E-mail: duzhang@mail.nankai.edu.cn.

TABLE I: Partial Trade Events in NASDAQ Dataset [15].

	<i>Ticker</i> [stock]	<i>Open</i>	<i>Volume</i>	<i>Date</i> [timestamp]
e_1	MSFT	329.67	60906	2023/05/26 10:55
e_2	AAPL	175.29	100686	2023/05/26 10:56
e_3	GOOG	125.55	82648	2023/05/26 10:57
e_4	AAPL	175.37	61517	2023/05/26 10:58
e_5	MSFT	330.85	109612	2023/05/26 11:03
e_6	GOOG	125.16	37939	2023/05/26 11:06
e_7	GOOG	125.08	29692	2023/05/26 11:15
e_8	GOOG	125.00	43324	2023/05/26 11:23

```

1 PATTERN SEQ (MSFT v1, GOOG v2, MSFT v3, GOOG v4)
2 FROM NASDAQ
3 USE skip-till-next-match
4 WHERE v1.open BETWEEN 326 AND 334
5 AND v2.open BETWEEN 120 AND 130
6 AND v3.open >= v1.open * 1.003
7 AND v4.open <= v2.open * 0.997
8 WITHIN 12 minutes
9 RETURN COUNT(*)

```

Listing 1: Complex event query statement Q_1 .

(i.e., $e_1 \rightarrow e_5$), and then GOOG's opening price decreases (i.e., $e_3 \rightarrow e_6$). Then, the investor may submit the query Q_1 (shown in List. 1) to event stores to retrieve the historical frequency of this pattern. If the pattern is infrequent, it may suggest a potential future scenario where either the opening price of GOOG will rise or that of MSFT will fall. Based on this insight, the investor can make strategic decisions to buy or sell stocks, aiming for maximum returns.

In Listing 1, **PATTERN** clauses depict the detected event pattern. In this pattern, the keyword **SEQ** is a temporal operator which indicates multiple events occurring chronologically; **FROM** clauses specify the queried primitive event set; **USE** clauses specify the selection strategy, i.e., the *skip-till-next-match*, which allows skipping the irrelevant events until identifying an event that matches previous results; **WHERE** clauses specify the predicate conditions that variables must satisfy; **WITHIN** clauses specify the maximum window distance between the first and last event in the pattern; and **RETURN** clauses specify the target output.

Typically, loading primitive events from event stores includes two methods: fullscan-based [2], [16]–[19] and index-based [20], as shown in Fig. 1: (1) Fullscan-based methods retrieve all events from event stores and feed these events into the evaluation engine, e.g., Non-deterministic Finite Automata (NFA) [17], [21], [22] for matching. However, they suffer from high latency due to costly disk I/O overhead and expensive matching overhead. Although several works [23], [24] attempt to pre-filter irrelevant events before sending events to the engine, the issue of costly disk I/O overhead persists, and the query latency still remains at a high level; (2) Index-based methods [20] utilize indexes to reduce I/O overhead

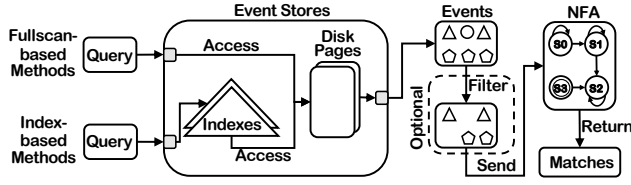


Fig. 1: Fullscan-based vs. Index-based methods.

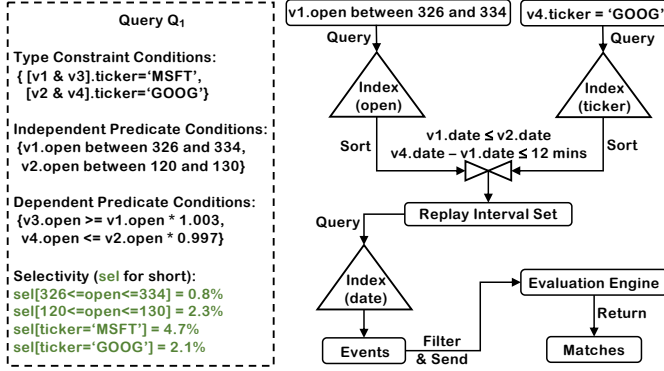


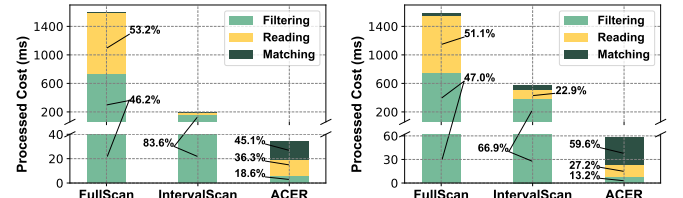
Fig. 2: Execution plan of prior SOTA index-based methods.

when reading events. They construct tree-based indexes for event attributes and greedily choose partial conditions with low selectivity to query. Since the events stored in the indexes may not be chronologically ordered, an additional sorting operation is vital to ensure the returned events are ordered. Notably, accessing tree-based indexes and sorting events incur significant I/O and sorting overhead, especially for the query with high predicate selectivity or long query window. As a result, prior index-based methods also suffer from high latency and may not be suitable for time-sensitive scenarios [25].

Specifically, we present the following example to understand the execution process of the prior index-based methods.

Example. Fig. 2 illustrates the execution plan of the state-of-the-art (SOTA) index-based method [20] for query Q_1 (three tree-like indexes have been built on ticker, open, and date columns). The method greedily selects the conditions with minimum selectivity from the type condition and independent predicate condition (IPC) set, using the built indexes to query these conditions. Note that the dependent predicate condition set is processed by the evaluation engine. During query, if a new condition can reduce the overall query cost, it is selected to query with indexes. Otherwise, it is skipped, and no further conditions are selected. In our example, conditions $v1.open$ BETWEEN 326 AND 334 and $v4.ticker='GOOG'$ are selected (preferring $v4$ rather than $v2$ as it can generate shorter replay intervals). Then, the method sorts the results, performs a join operation to generate the replay intervals that potentially contain matches, and queries the index of the date column to find these events whose timestamps are within these replay intervals. Finally, it filters these events again and sends the filtered events to the evaluation engine for matching.

Next, we conduct experiments to analyze the query cost, aiming to identify the factors that lead to high query latency. Typically, the total query cost can be decomposed into three main components: filtering, reading, and matching costs. Specifically, when indexes are available, the filtering cost includes both the overhead of loading relevant index structures into memory and the overhead of querying these



(a) Cost breakdown of Q_1

(b) Cost breakdown of Q_2

Fig. 3: Cost breakdown of Q_1 and Q_2 on NASDAQ dataset.

indexes in memory to prune irrelevant events. Otherwise, the filtering cost refers to the overhead of pruning irrelevant events in memory using IPC and window conditions. The reading cost denotes the cost of retrieving events from disk, while the matching cost denotes the cost of matching in evaluation engines. Then, we evaluate the breakdown cost of two queries, Q_1 and Q_2 , on NASDAQ dataset (each query runs 10 times, and Q_2 has a higher predicate selectivity and a longer window size than Q_1). The results are shown in Fig. 3. Here, FullScan and IntervalScan represent the SOTA fullscan-based [23] and index-based methods [20], respectively. ACER is our solution.

IntervalScan has a lower reading cost than FullScan, but it incurs significant filtering cost due to extra index accessing and result sorting (sorting cost accounting for about 6~11% of filtering cost). Besides, it still has a high reading cost for Q_2 since it first identifies the time ranges that may contain matches and then loads the events within these ranges from disk before secondary filtering, causing unnecessary reading cost. Overall, in designing index structures to lower the latency of CER, we face three primary challenges: (C1) reducing I/O overhead when reading indexes; (C2) ensuring that queried events are retrieved in chronological order without extra sorting; and (C3) avoiding unnecessary disk access in indexes or events.

To address C1, we first group events of the same event type into a cluster. Then, we only need to store event type information once per cluster, which reduces storage redundancy. Besides, we organize the index fields (i.e., timestamp and RID) by column and compress them to reduce I/O overhead when reading indexes. To address C2, we construct Range Bitmaps for attribute values and ensure that the events in each cluster are chronologically ordered. Since the output order of the bitmap is aligned with the construction order, we can always generate ordered query results for a specific event type by sequentially merging the query results of each index block (even if events arrive out-of-order), thus avoiding sorting overhead. Lastly, to address C3, we query all IPCs and type conditions since our indexes provide a low query cost. Besides, to avoid unnecessary disk access in indexes and events, we propose a two-phase filtering algorithm (a.k.a. window-wise filtering) based on the window condition. Its key intuition is that: (i) if an index block's time range cannot overlap with the replay intervals, it can be skipped; and (ii) for events whose timestamps fall within a replay interval, if none satisfy a variable's type condition and IPCs, then these events cannot form matches, and we can skip accessing them.

Overall, our key contributions are summarized as follows:

- We propose an index structure named ACER to accelerate CER. It achieves a low space overhead by eliminating redundant type information storage and compressing the timestamp and RID information of index blocks. Besides, we propose an enhanced version of ACER (ACER-E),

which optimizes index block read/write operations and the variable query order to improve overall performance.

- We propose a window-wise filtering algorithm to prune the index blocks without matches and avoid loading the events that cannot be involved in matching. This algorithm effectively reduces unnecessary disk I/O and can be extended to filtering events for the query with Kleene or negation operators.
- We conduct extensive experiments to verify the performance of ACER and ACER-E on the six real-world and synthetic datasets. The experimental results show that ACER and ACER-E reduce the query latency by up to one order of magnitude compared with SOTA techniques.

II. PRELIMINARIES

A. Complex Event Query Template

Listing 2 shows the complex event query template we support. It contains six parts: event pattern, event set, selection strategy, predicate condition set, query window, and expected output. Next, we introduce the six parts sequentially.

1) *Event Pattern*: It provides the temporal sequential relationship of multiple events. Typically, its grammar is

```
(1) S -> PatternExpr
(2) PatternExpr -> SEQ(PatternExpr, PatternExpr)
(3) PatternExpr -> AND(PatternExpr, PatternExpr)
(4) PatternExpr -> OR(PatternExpr, PatternExpr)
(5) PatternExpr -> PatternExpr, PatternExpr
(6) PatternExpr -> typeCondition variableName
```

where SEQ, AND, and OR are temporal operators. SEQ operator indicates multiple events occurring chronologically. AND operator indicates all events must occur, regardless of event occurrence order. OR operator indicates at least one event occurring among multiple events.

Note that for an event pattern with OR operator, we can decompose it to multiple event patterns without OR operator [23], [26], [27]. Additionally, for queries that contain other special operators such as Kleene and negation operators [28], to the best of our knowledge, they can be transformed into queries containing only SEQ and AND operators for filtering (see §III-E). Thus, unless otherwise stated, the queries discussed in this paper only contain SEQ and AND operators.

2) *Event Set*: It contains N primitive events $\{e_1, \dots, e_N\}$, each sharing a same schema $(Type, Attr^1, \dots, Attr^d, Ts)$, where $Type$ specifies its type, $Attr^i$ is the i -th attribute of events, and Ts refers to an event occurrence timestamp. When the events in the set are chronologically ordered (i.e., $e_i.Ts \leq e_j.Ts$ for any $i < j$, where $e_i.Ts$ denotes the timestamp of e_i), they can directly send to the evaluation engine for matching. Otherwise, an additional sort operation based on timestamp is vital before sending them to the evaluation engine.

3) *Selection Strategy*: It specifies the rules by which the evaluation engine selects events for matching. There are three match strategies are available [2], [17], [29]: *strict-contiguous*, *skip-till-next-match*, and *skip-till-any-match*. The first one requires that adjacent events in the pattern must occur continuously. The second one allows the engine to skip irrelevant events until it finds the event that matches previous results. The last one allows the engine to skip any events, regardless of whether it can match the previous results.

```
1 PATTERN event_pattern
2 FROM event_set
3 USE selection_strategy
4 WHERE predicate_condition_set
5 WITHIN query_window
6 RETURN expected_output
```

Listing 2: Complex event query template.

Since the last two selection strategies have fewer matching restrictions and are more robust, they are extensively applied in diverse scenarios [26], [30]–[33]. Thus, this paper mainly addresses the queries using the last two selection strategies.

4) *Predicate Condition Set*: It is composed of independent and dependent predicate conditions. Specifically, the independent predicate condition (IPC) specifies the attribute value property of events and can be evaluated on a single event [23]. The dependent predicate condition (DPC) specifies the attribute value relationship between multiple events, and it must be evaluated on multiple events [23].

5) *Query Window*: It defines the maximum distance between any two events in the event pattern. There are two types of distance: (1) count-based, which limits the number of intervening events between any two events; and (2) time-based, which restricts the maximum difference in timestamps between two events. This paper focuses on the predominant time-based query window. Notably, extending our index structures to support count-based distance is easy as count-based is a special case of time-based when the event arrival rate is fixed.

6) *Expected Output*: It refers to a specific result format that the user expects the evaluation engines to produce, e.g., the full matches, the number of matches, etc.

B. Bitmap Structures

Before introducing the detailed bitmap structures, we first clarify their role in our index design. Note that variables in complex event queries are often constrained by multi-attribute range predicates, and retrieving events in chronological order eliminates the need for additional sorting before matching. Thus, the ideal index backbone needs to simultaneously support efficient range filtering and ordered result generation.

The base-2 bit-sliced range-encoded bitmap (Range Bitmap for short) [34], [35] satisfies these requirements. *It enables fast evaluation of range predicates via bitwise operations while inherently preserving event order, thereby producing ordered results without extra sorting.* Moreover, it stores original attribute values losslessly, guaranteeing zero false positives during range queries. In contrast, lossy or approximate bitmap schemes [36]–[38] may introduce false positives, i.e., returning rows that do not satisfy the range constraints. As a result, these methods require secondary refinement that fetches full rows from disk to eliminate false positives, incurring additional I/O overhead. Furthermore, excessive false positives would undermine the effectiveness of our filtering algorithm. Thus, Range Bitmap is a suitable backbone for our index design.

Having clarified Range Bitmap as our index backbone, we now detail its internal structure and query mechanism. Fundamentally, Range Bitmap is organized as a bit-sliced, range-encoded representation in which each slice corresponds to one bit position of the binary encoding of attribute values. As exemplified in Fig. 4, for an item set $\{7, 1, 5, 0, 4, 5, 2, 1\}$,

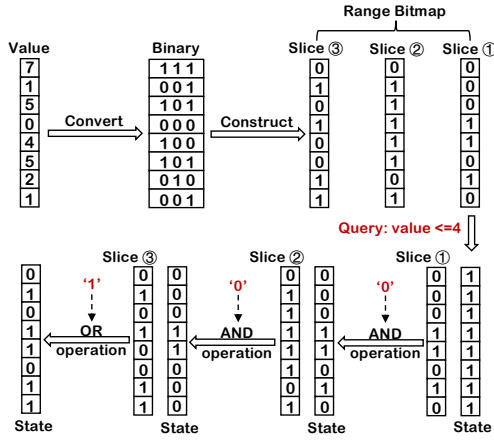


Fig. 4: Construction and query process of Range Bitmap.

its Range Bitmap consists of three sliced bitmaps. The values of each row of this Range Bitmap are the bitwise inverse of the input values (a more detailed construction and query process can be seen in [39]). To identify locations where the item value is no greater than a query threshold, Range Bitmap first creates an initial bitmap *state*, with all bit values set to 1¹. Next, it converts the threshold into binary, denoted as *t*. For the *i*-th bit value *v* in *t*, if *v* is equal to 1, an OR operation is performed on *state* and the *i*-th slice bitmap, and the result is updated to *state*. Otherwise, an AND operation is performed on *state* and the *i*-th slice bitmap, and the result is updated to *state*. By performing the OR/AND operation from the lowest bit to the highest bit of *t*, we obtain a bitmap indicating which locations (*i.e.*, the locations marked ‘1’ in the bitmap) have item values not greater than the query threshold. An example of finding positions with item values no greater than 4 in the above constructed Range Bitmap is shown in Fig. 4.

Notably, by negation or intersection operation, Range Bitmap can be extended to support all types of range queries. Additionally, when processing multiple IPCs for a given variable, Range Bitmap can efficiently return query results via an AND bitwise operation. However, *even if the selectivity of an IPC is low (e.g., 0.1%), Range Bitmap still needs to access all bits to determine which values satisfy the given IPC*. In §III-D, we will avoid this drawback via the clustering, data synopsis, and window-wise filtering techniques.

III. DESIGN OF ACER

Before diving into the detailed structures and algorithms, we first summarize the design highlights of ACER. ACER mitigates the dominant I/O and sorting overheads in CER through four core ideas: ❶ type-aware clustering and compression to compactly store same-type events and reduce index footprint; ❷ lightweight synopsis metadata that records per-cluster time and attribute ranges to prune irrelevant index blocks early; ❸ chronologically organized Range Bitmaps to support efficient range filtering while directly producing ordered results, eliminating sorting; and ❹ window-wise filtering to progressively shrink candidate intervals and avoid unnecessary accesses to indexes and events. Together, these techniques enable coarse-to-fine pruning, substantially reducing costly I/O and sorting overhead and thereby achieving low-latency CER.

¹Note that Range Bitmap can evaluate range predicates horizontally or vertically. Here, we introduce the latter method to facilitate readers’ understanding.

TABLE II: Notations

Symbol	Description
e_i	The i -th event stored in the index block
$e_i.Attr^k$	The k -th attribute value of e_i
$\mathbf{A}_j^k$	Attribute set of $Attr^k$ in the j -th cluster
v_i	The i -th variable in the event pattern
Θ_i^{IPC}	IPC set bound to variable v_i
ξ	Number of variables in the event pattern
$e_i.Ts$	Timestamp of event e_i
δ	Time interval that may contain matches
w	Window size of complex event query
ST	Synopsis table of ACER

The remainder of this section is organized as follows. We first present the ACER structure overview, followed by its insertion, deletion, and update operations. Then, we introduce the query process and discuss how ACER handles queries with Kleene and negation operators. Finally, we analyze the query complexity. Frequently used notations are listed in Table II.

A. ACER Structure Overview

ACER structure is shown in Fig. 5. It mainly contains four components: Page, Buffer Pool, Index Block, and Synopsis Table. Specifically,

1) *Page*: It stores the entire event information persistently in a row format. When an event arrives, we append it to the Page and return its physical record identifier (RID for short, which is a composite of page number and offset position, representing the physical storage location). Notably, we use a row-oriented storage format instead of a columnar format, as row storage provides higher write throughput and returns results without attribute concatenation operations.

2) *Buffer Pool*: It temporarily holds events awaiting indexing, with a fixed capacity. It consists of multiple buffers, each storing partial event information, including its timestamp, RID, and attribute values pending indexing. Notably, events within the same buffer share identical event types. When the buffer pool reaches its capacity, if the events are not already ordered by timestamp, an additional sorting is performed to ensure chronological order. Subsequently, serialization occurs, *i.e.*, converting the attribute values, timestamp list, and RID list into byte arrays and writing these arrays to index files. After serialization, the Synopsis Table is updated, and the buffers that have been processed are cleared.

3) *Index Block*: It includes Range Bitmaps (RB) of event attributes, a timestamp list (Ts List for short), and an RID list. Notably, in the Index Block, event information of the same type are grouped in multiple continuous byte segments termed a cluster. Besides, the Index Block stores compressed values rather than raw values. Specifically, (1) for the i -th event e_i stored in the Index Block (assuming it belongs to the j -th cluster within this block), the value inserted into RB for attribute $Attr^k$ is $e_i.Attr^k - \min\{\mathbf{A}_j^k\}$, where $\mathbf{A}_j^k$ denotes the attribute set of $Attr^k$ in the j -th cluster; and (2) similarly, both the timestamp list and RID list in the Index Block also store fixed-length (32 bits) delta values.

4) *Synopsis Table*: It logs synopsis information for each cluster across different Index Blocks. The cluster synopsis information includes: *Index Block ID* (an identifier of the index block where the cluster is located); *Cluster ID* (an identifier for the cluster); *Cluster region* (a location range that indicates events belonging to this cluster); *Minimum* &

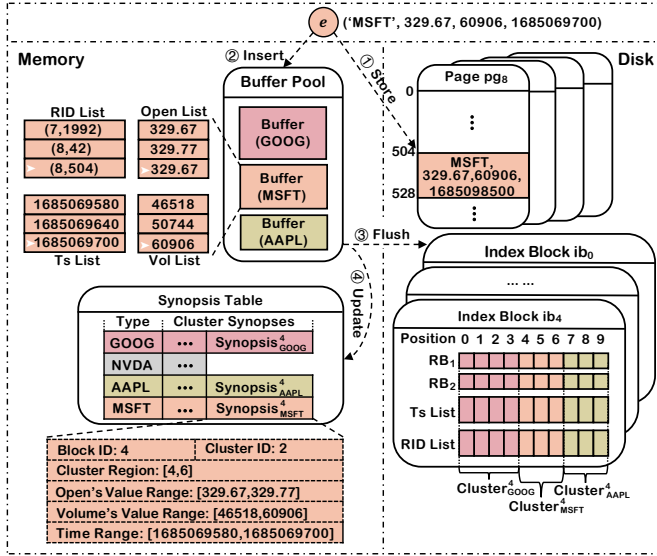


Fig. 5: ACER structure and its insertion example.

maximum attribute values within this cluster; and *Time range* (a time span that records the minimum and maximum event occurrence timestamps for this cluster). This information helps to skip unnecessary index block accesses during queries.

Note that ACER cannot directly index floating-point or string attributes. This limitation can be mitigated through two practical strategies: ① Floating-point values can be scaled to integers by multiplying by 10^s , where s denotes the maximum number of decimal places. This approach is effective for attributes with fixed precision (e.g., temperature or price). However, users must predefine s according to the required precision: an excessively large s may cause the scaled value to exceed the representable range of a 64-bit signed integer, resulting in overflow, while a small s may lead to precision loss. ② String values can be encoded into integers using dictionary encoding. In most real-world event sets, string attributes (e.g., status codes, district IDs, or categories) are categorical with low cardinality, making dictionary encoding highly effective. For high-cardinality strings (e.g., UUIDs or free-text fields), however, dictionary encoding introduces substantial overhead with little benefit. In such cases, it is more practical to avoid indexing these columns.

B. Insertion

When an event arrives, we store it on the Page and obtain its RID. Next, we extract the event's type, timestamp, and indexed attribute values, and append them to the targeted buffer. Once the insertion process is complete, we check whether the Buffer Pool has reached its capacity. If yes, we first sort the data to ensure it is chronologically ordered in all buffers and serialize the data to disk for generating a new Index Block. Notably, when a buffer finishes the flush operation, we generate the buffer's synopsis information, add it to the Synopsis Table, and clear the buffer. If the Buffer Pool is not full or the serialization is finished, we return "true" to indicate a successful insertion. We omit the insertion algorithm to save space.

Example. Fig. 5 shows a running insertion example on NASDAQ dataset. When an event $e = ('MSFT', 329.67, 60906, 1685069700)$ arrives, ACER first appends it on the Page and returns its RID, denoted by $rid = (8, 504)$. Second,

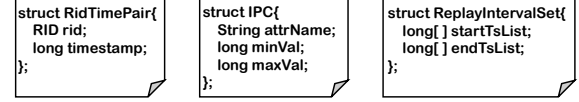


Fig. 6: Structure declarations.

ACER inserts rid , attribute values, and the timestamp of e into the buffer 'MSFT'. At this time, the Buffer Pool reaches its capacity. Third, ACER sequentially processes all buffers. In our example, the buffer 'MSFT' is the second processed buffer, and its synopsis information $Synopsis_{MSFT}^4$ is presented in Fig. 5. Since the data within the buffer 'MSFT' is chronologically ordered, ACER directly appends the data to the open value list, volume value list, Ts list, and RID list. Fourth, ACER updates the Synopsis Table for this buffer and subsequently clears the buffer's content. Once processing all buffers, ACER (1) generates the serialized Range Bitmaps RB_1 and RB_2 for the open and volume attributes, and (2) compresses the Ts list and RID list to obtain their serialized byte arrays. Finally, ACER appends these arrays to the index file, finishing the flush operation.

C. Deletion and Update

Generally, when deleting or updating an event e , we have to scan all pages to locate e . Although the Synopsis Table can help skip unnecessary page accesses, the process of deletion or update remains prohibitively expensive. The reason is that once the information of deleted or updated events is no longer in the Buffer Pool, we may inevitably read the entire Index Block from disk for refactoring. In our setting, each Index Block exceeds 100 KB, making in-place updates and deletions prohibitively expensive.

To alleviate the high cost of in-place deletions and updates, we adopt an out-of-place modification strategy. We consider two cases: (1) *Events with primary key*: If events have a primary key, then for deletion operations, we append a new event with the same primary key and mark all non-primary-key attributes as logically null, serving as a tombstone. During query processing, multiple versions of the same primary key may be retrieved; in such cases, only the latest version is considered visible. If the latest version is marked as logically null, the event is treated as deleted. For update operations, we logically delete the old version by appending a tombstone event and then insert the updated event as a new version. (2) *Events without primary key*: For such events, we use a bitmap to indicate deletions. When deleting an event, the corresponding bit is set to 1. Updating an event involves deleting the expired version and then inserting the updated one. Notably, when deletions or updates occur, we need to remove expired events based on primary keys or the deletion bitmap before sending the queried events to the evaluation engine. Overall, this out-of-place modification strategy achieves latency for deletions and updates comparable to that of insertions, making it highly suitable for write-intensive workloads.

D. Query

Since ACER processes queries at variable granularity, we first introduce how ACER queries events that hold a single variable's type condition and IPCs (the DPCs are processed by the evaluation engine). Then, we will introduce how ACER handles complex event queries with multiple variables.

Algorithm 1: Query process of a single variable

Input: Variable v_i , the IPC set Θ_i^{IPC} bound to v_i , and the Synopsis Table ST .

Output: $RidTimePair$ set that holds v_i 's type condition and Θ_i^{IPC} .

```

1 String  $type \leftarrow$  the event type bound to  $v_i$ ;
2 int []  $blockIDs \leftarrow ST.getAllBlockIds(type, \Theta_i^{IPC})$ ;  $\triangleright$  retrieve Index
  Block IDs that satisfy the type condition and IPC set from  $ST$ 
3  $regions \leftarrow ST.getAllClusterRegion(type, \Theta_i^{IPC})$ ;  $\triangleright$  obtain the
  cluster region of each Index Block corresponding to  $type$  from  $ST$ 
4  $RidTimePair[] pairs$ ;  $\triangleright$  store the intermediate results
5 for  $j$  in  $range(blockIDs.length)$  do
6    $id = blockIDs[j]$ ;  $\triangleright$  obtain current Index Block id
7   IndexBlock  $ib \leftarrow$  obtain the specified Index Block based on  $id$ ;
8    $regionBitmap \leftarrow$  create a region bitmap for IndexBlock  $ib$ 
     based on  $regions[j]$ ;  $\triangleright$  obtain the bitmap that satisfies the
     type conditions of  $v_i$ 
9    $Bitmap bmp_0 = regionBitmap$ ;  $\triangleright$  set initial bitmap
10  foreach  $ipc$  in  $\Theta_i^{IPC}$  do
11    RangeBitmap  $rb \leftarrow$  obtain the attribute Range Bitmap
      from the Index Block  $ib$  by  $ipc.attrName$ ;
12     $bmp_1 = rb.rangeQuery(ipc.minVal, ipc.maxVal)$ ;
13     $bmp_0 = bmp_0 \& bmp_1$ ;  $\triangleright$  intersect two bitmaps
14   $curRidTimePairs \leftarrow$  find RID and timestamp pairs of
     corresponding positions from Ts list and RID list of Index
     Block  $ib$  using bitmap  $bmp_0$ ;  $\triangleright$  query Index Block  $ib$ 
15  Merge  $curRidTimePairs$  to  $pairs$ ;  $\triangleright$  merge two ordered lists
16  $bufferPairs \leftarrow$  access  $BufferPool[type]$  to find events that satisfy
      $\Theta_i^{IPC}$ , and return ordered  $RidTimePair$  results;  $\triangleright$  query buffer
17 Merge  $bufferPairs$  to  $pairs$ ;  $\triangleright$  merge two ordered lists
18 Remove expired events based on the primary key or deletion marker;
19 return  $pairs$ .
```

Before introducing, we give three structure declarations shown in Fig. 6. Here, $RidTimePair$ consists of the timestamp and RID of an event, which serves as an intermediate query result for a single variable (without using RID as the result since we need the timestamp to filter further). IPC defines an independent predicate condition (we have equivalently converted float-point or string values to long values). $ReplayIntervalSet$ (RIS for short) is a set containing several time intervals that may contain matches, each defined as a period starting and ending at specific timestamps. Notably, RIS is always ordered, and it includes several functions, e.g., $checkOverlap$, $updateIntervals$, and $filterPairs$. These functions will be explained later.

1) *Query Process of a Single Variable:* Assume that the event pattern of a given query contains ξ variables $v_1, \dots, v_\xi$, and the IPC set bound to v_i is denoted by Θ_i^{IPC} . Algorithm 1 illustrates the query process of a single variable.

To retrieve the $RidTimePair$ set that holds the type condition and IPCs bound to v_i , we first obtain the event type associated with v_i (Line 1). Next, we access the Synopsis Table, find all Index Blocks that contain the event type and whose attribute value ranges overlap with all IPCs, and return their block IDs (Line 2). Afterwards, we get the cluster region of each Index Block corresponding to the event type from the Synopsis Table (Line 3). Then, we read the Index Blocks from the disk by Index Block IDs, query the Range Bitmaps to identify the events that satisfy Θ_i^{IPC} (Line 5-13). Later, we obtain the $RidTimePair$ results based on the bitmap and merge them into the result set because the events of the same type in each Index Block are chronologically ordered (Line 14-15). Since the Buffer Pool also holds the events, we still need to find the events that satisfy Θ_i^{IPC} from the corresponding buffer and

Algorithm 2: Query process of complex event query

Input: A complex event query without OR operator.

Output: Matches.

```

1 Estimate the overall selectivity of each variable in the query;
2  $v_{s1}, \dots, v_{s\xi} \leftarrow$  sort variables based on their overall selectivity;
3  $\Theta_{s1}^{IPC} \leftarrow$  obtain the IPC set of  $v_{s1}$ ;
4  $minPairs \leftarrow$  put the parameters  $v_{s1}$  and  $\Theta_{s1}^{IPC}$  into Algorithm 1
  to obtain the  $RidTimePair$  results;  $\triangleright$  start the first-phase filtering
5  $Map < String, RidTimePair[] > varPairsMap$ ;  $\triangleright$  create map
6  $varPairsMap.put(v_{s1}, minPairs)$ ;  $\triangleright$  store the query results of
   $v_{s1}$  in the hash map
7  $RIS \leftarrow$  generate the replay interval set based on  $minPairs$ ;
8 foreach variable  $v_i$  in  $\{v_{s2}, \dots, v_{s\xi}\}$  do
9    $type \leftarrow$  obtain the event type bound to the variable  $v_i$ ;
10   $timeRanges \leftarrow ST.getAllBlockTimeRanges(type, \Theta_{s1}^{IPC})$ ;  $\triangleright$ 
    retrieve the time ranges of specific clusters from  $ST$ , based on
    the provided  $type$  and  $\Theta_{s1}^{IPC}$ 
11  bool []  $overlaps \leftarrow RIS.checkOverlap(timeRanges)$ ;  $\triangleright$ 
    check which Index Blocks' time ranges overlap with  $RIS$ 
12  int []  $blockIDs \leftarrow ST.getAllBlockIds(type, \Theta_{s1}^{IPC}, overlaps)$ ;  $\triangleright$ 
    identify the Index Block IDs that need to be accessed
13   $curPairs \leftarrow$  query the  $RidTimePair$  results that satisfy the
    type condition and IPCs of  $v_i$  from the above Index Blocks
    and BufferPool;  $\triangleright$  same as Line 5-18 in Algorithm 1
14   $RIS.updateIntervals(curPairs)$ ;  $\triangleright$  via Observation 1
15   $varPairsMap.put(v_i, curPairs)$ ;  $\triangleright$  store the query results
    of  $v_i$  to wait secondary filtering
16  $Event[] events$ ;  $\triangleright$  start the second-phase filtering
17 foreach variable  $v_k$  in  $\{v_1, \dots, v_\xi\}$  do
18   $pairs_k = varPairsMap.get(v_k)$ ;  $\triangleright$  obtain  $RidTimePairs$ 
19   $filteredPairs_k = RIS.filterPairs(pairs_k)$ ;  $\triangleright$  remove the
     $RidTimePairs$  whose timestamps are not within  $RIS$ 
20   $varPairsMap.put(v_k, filteredPairs_k)$ ;  $\triangleright$  update the
     $RidTimePairs$  of  $v_k$ 
21  $allPairs \leftarrow$  merge all  $RidTimePairs$  in  $varPairsMap$  based on
    their timestamps and remove duplicate  $RidTimePairs$ ;
22  $events \leftarrow$  retrieve events by  $allPairs$ ' RIDs;
23 return  $EvaluationEngine.extractMatches(events)$ .
```

merge query results into the result set (Line 16-17). Notably, if events arrive out-of-order, we need to sort the query results of the buffer before merging them into the result set. Finally, we remove expired events based on the primary key or deletion marker (Line 18) and return the query result set (Line 19).

2) *Query Process of Multiple Variables:* We first present a pruning observation in matching, and then propose a window-wise filtering to avoid unnecessary disk I/O when processing a given complex event query with multiple variables.

Empirically, for an event e_j that holds the type condition and IPCs of variable v_i , if e_j can form a match, then other events in this match must occur in the following replay interval:

$$\delta = \begin{cases} [e_j.Ts, e_j.Ts+w], & v_i \text{ is the head variable,} \\ [e_j.Ts-w, e_j.Ts], & v_i \text{ is the tail variable,} \\ [e_j.Ts-w, e_j.Ts+w], & \text{otherwise,} \end{cases} \quad (1)$$

where w denotes the window size, the head variable denotes that the event corresponding to this variable is always the first event in matches, while the tail variable denotes that the event corresponding to this variable is always the last event in matches. For example, for the pattern $SEQ(type1\ v1, type2\ v2)$, its head variable is $v1$, tail variable is $v2$, however, for the pattern $AND(type1\ v1, type2\ v2)$, there are no head or tail variables. Then, we have the following pruning observation:

Observation 1. *Given a query without the OR operator, for the events whose timestamps are within a replay interval, if*

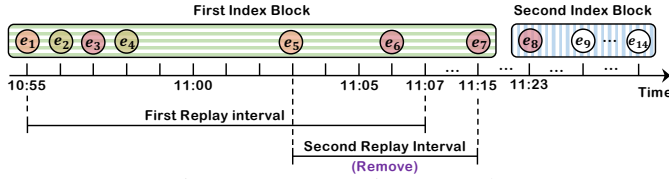


Fig. 7: Query running example.

none of them can hold a variable's type condition and IPCs (note that events cannot be reused by multiple variables), then this replay interval has no matches for this query.

Inspired by Observation 1, we propose a window-wise filtering algorithm to avoid unnecessary disk I/O. Our key intuition is: (1) after obtaining the RIDTimePair results of the first queried variable, we can generate a replay interval set RIS based on the query results. Then, if an Index Block's time range cannot overlap with RIS , it can be skipped as it cannot contain any matches; and (2) after obtaining the query results of all variables, we can utilize Observation 1 to remove irrelevant replay intervals from RIS . After that, when querying related events from a disk, we can filter these events whose timestamps are not within RIS , which effectively avoids accessing irrelevant events from the disk.

Algorithm 2 outlines the process of executing a complex event query with multiple variables in ACER, which employs a two-phase filtering approach to minimize unnecessary disk I/O. Before starting the first-phase filtering (Line 4-15), it first estimates the overall selectivity of each variable (Line 1) and sorts them based on selectivity (Line 2). Second, it queries the RidTimePair results of the variable with minimum overall selectivity and stores the query results in a hash map (Line 4-6). Third, it generates the replay interval set RIS for the first processed variable by Equation 1 (Line 7). Fourth, it queries the RidTimePair results of other variables (Line 8-15). To avoid unnecessary index access, it verifies whether an Index Block's time region overlaps with RIS (Line 10-11). If yes, the Index Block may contain the events participating in matching, and it needs to be queried. Otherwise, the Index Block can be skipped when accessing. The `checkOverlap` function achieves the overlap check and returns a boolean list to mark all Index Blocks that must be read. Then, ACER accesses these Index Blocks (according to the boolean list) and the Buffer Pool to obtain the query results (Line 12-13). Once obtaining the query results of a variable, it prunes the replay intervals that cannot contain matches by Observation 1 (Line 14). Finally, it puts the query results of the current variable into the hash map.

In the second-phase filtering (Line 16-22), ACER first removes the RidTimePair results whose timestamps are not within RIS using the `filterPairs` function (Line 17-20). Next, it merges all RidTimePairs in the hash map based on their timestamps and removes duplicate RidTimePairs (Line 21). Then, it retrieves events from the disk using RIDs (Line 22). Lastly, it sends remaining events into the evaluation engine to obtain matches and returns these tuples (Line 23).

Example. Suppose an Index Block only stores 7 events. When processing Q_1 on the event set in Table I (note that we add 6 additional events $e_9 \sim e_{14}$ of non MSFT and GOOG types to the event set), ACER first calculates the overall selectivity of each variable and identifies that v_1 has the

minimum selectivity. Second, it uses Algorithm 1 to obtain query results $\{(rid_1, 10:55), (rid_5, 11:03)\}$ for v_1 , where rid_i denotes the RID of e_i . Third, it generates the replay interval set $RIS = \{[10:55, 11:07], [11:03, 11:15]\}$ by Equation 1 (as shown in Fig. 7). Fourth, it continues to query the results of other variables. Notably, before querying an Index Block, ACER verifies whether the Index Block's time range overlaps with RIS . Clearly, the second Index Block cannot overlap with RIS . Thus, other variables can skip the second Index Block during query. Besides, when obtaining the query results of v_3 , i.e., $\{(rid_1, 10:55), (rid_5, 11:03)\}$, since e_5 has been used by v_1 , the second replay interval is removed by `updateIntervals` function based on Observation 1. After finishing the second-phase filtering, the merged RidTimePair results are $\{(rid_1, 10:55), (rid_3, 10:57), (rid_5, 11:03), (rid_6, 11:06)\}$. Then, ACER retrieves events from the disk page based on these RIDs. Without deletion and update, ACER sends $\{e_1, e_3, e_5, e_6\}$ to the evaluation engine for matching.

E. Special Operator Processing

When the query contains special operators, i.e., Kleene Plus (KP for short, which allows events to occur one or multiple times), Kleene Star (KS for short, which allows events to occur zero or multiple times), or Negation (NEG for short, which cannot allow events bound by it to occur between two specified events), ACER can still assist in filtering events. Specifically,

(1) If the query Q contains the KP operators, we first remove the KP operators of Q to create a new query Q' . Then, we submit Q' to ACER to obtain filtered events. Finally, the filtered events and the original query are sent to the evaluation engine for matching. For example, for a query with the pattern $SEQ(A\ a, KP(B\ b), C\ c)$, we will submit the pattern $SEQ(A\ a, B\ b, C\ c)$ to ACER for filtering.

(2) If the query Q contains the KS or NEG operators, we first remove the variables bound by KS or NEG operators and their predicate conditions from Q to create a new query Q'' . Second, we submit Q'' to ACER to acquire the preliminary filtered events and the final replay interval set RIS . Third, we query events that satisfy the type condition and IPCs of the variables bound by KS or NEG operators fall within RIS . Lastly, we merge these events with the previously filtered events and send them to the evaluation engine for matching.

In §V-C5, we will evaluate the performance improvement when using ACER to handle these special operators.

F. Algorithm Complexity Analysis

Theorem 1. The space complexity of Algorithm 2 is always $O(N)$; when events arrive in order, the time complexity of Algorithm 2 is $O(\xi N)$; for out-of-order event arrivals, the time complexity becomes $O(\xi N + (\xi - 1) \lfloor \frac{N}{T_B} \rfloor \log \lfloor \frac{N}{T_B} \rfloor)$, where N is the number of events, ξ is the number of variables, and T_B is the capability of the Buffer Pool.

Proof. The space complexity of Algorithm 2 is noticeable, as it stores all RidTimePairs of events in the worst-case scenario. Now, we discuss the time complexity in two cases:

Case 1: When events arrive in order, obtaining the RidTimePair results from the first processed variable takes $O(N)$ since the events in both the Index Block and Buffer Pool are

chronologically ordered. Maintaining consistency between the Index Block read order and the generation order allows direct appending of current `RidTimePair` results to the previous result set. For other variables, the ordered replay interval set and time range list enable the `checkOverlap` function to run in linear time. Besides, since the `RidTimePair` results of each variable are ordered, the functions `updatedIntervals` and `filterPairs` also operate in linear time. Thus, the time complexity of querying other variables is $O(N)$, thereby the overall time complexity is $O(\xi N)$.

Case 2: When events arrive out-of-order, the time complexity of obtaining the `RidTimePair` results from the first processed variable is $O(N + T_B \log T_B)$ since we need to sort the query results in the Buffer Pool. For other variables, the time range list for a given event type is unordered. The function `checkOverlap` will sort the unordered time range list, which results in $\lfloor \frac{N}{T_B} \rfloor \log \lfloor \frac{N}{T_B} \rfloor$ complexity. Thus, the time complexity of querying other variables is $O(N + T_B \log T_B + \lfloor \frac{N}{T_B} \rfloor \log \lfloor \frac{N}{T_B} \rfloor)$. As a result, the overall time complexity of Algorithm 2 is $O(\xi N + (\xi - 1) \lfloor \frac{N}{T_B} \rfloor \log \lfloor \frac{N}{T_B} \rfloor)$, where we omit $\xi T_B \log T_B$ since T_B is a fixed value. $\square$

Although the worst-case space complexity of Algorithm 2 is $O(N)$ (here each variable's `RidTimePairs` is stored in the hash map), its practical memory footprint is carefully controlled:

- ① Window-wise filtering and synopsis-based pruning substantially reduce the number of intermediate results by discarding irrelevant index blocks and `RidTimePairs` early, so only a small fraction of `RidTimePairs` are stored in practice.
- ② Each `RidTimePair` consists of two fields: an 8-byte timestamp and a 6-byte RID. Then, multiple `RidTimePairs` are compressed in memory using efficient delta encoding for each field to reduce space consumption. As a result, the average footprint of a `RidTimePair` is typically only 8 bytes.
- ③ When the candidate `RidTimePairs` exceed a predefined memory budget (e.g., 200MB), sorted `RidTimePairs` are temporarily written to disk. Since `RidTimePairs` are produced in chronological order, they can be merged efficiently via external merge sort, enabling efficient filtering in Algorithm 2.

IV. OPTIMIZATION OF ACER

In this section, we propose an enhanced version of ACER (ACER-E), which optimizes index block read/write operations and the variable processing order to improve its query performance further. Specifically, these optimizations include:

A. Fine-grained Index Block Access

Accessing the Index Block at the block level may cause unnecessary I/O overhead. For example, (i) when obtaining the query result of a variable, we only access one cluster from each Index Block, however, we need to decompress the entire Ts and RID lists; and (ii) if Range Bitmaps are built for four attributes but only two are needed to query when processing a variable's IPCs, loading the entire Index Block from disk leads to unnecessary I/O. To address these issues, we first propose an optimized Index Block layout as shown in Fig. 8, where D denotes the number of indexed attributes and G denotes the number of clusters stored. Its key insight is to compress the Ts and RID lists at the cluster-granularity

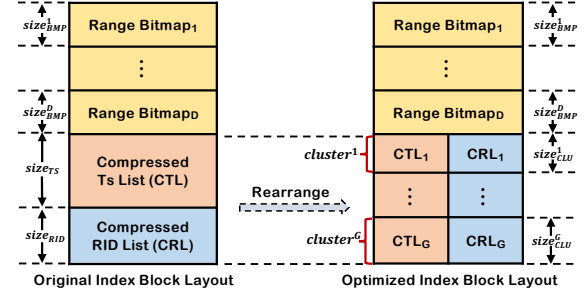


Fig. 8: Optimization of Index Block layout.

rather than the block-granularity. With the optimized layout, we can directly read and decompress only the relevant cluster, avoiding unnecessary decompression of the entire Index Block. Besides, we maintain the storage positions of Range Bitmaps for each Index Block, and then we can load Range Bitmaps on demand when processing range queries based on IPCs.

B. High-ratio Block Compression

Typically, attribute values of the same event type have a slight fluctuation within the cluster of the Index Block. As a result, the space overhead of a single attribute value in Range Bitmaps is about 1 to 2 bytes. In contrast, since each Index Block typically has a large capacity (usually 64KB), the maximum delta values for timestamp and RID in each cluster can become quite large (which is why we use 32 bits to store delta values in §III-A2). To alleviate the high storage overhead, we propose the Delta-Zigzag-Simple8B for compressing the timestamp and RID lists. Specifically, we first compute the delta between the current and previous raw values, then use Zigzag encoding [40] to handle potential negative deltas. Finally, we apply Simple8B encoding [41] to compress the Zigzag-encoded values, storing the compressed data within the Index Block.

C. Improved Variable Query Order

Algorithm 2 initially selects the variable with the lowest selectivity as the first processed variable. However, we observe that this variable may not yield the shortest interval length by Equation 1. Generally, a shorter interval length is more beneficial as it allows for filtering a greater number of index blocks. Thus, we give the following theorem to calculate the expected interval length.

Theorem 2. When the events holding the type condition and IPCs of variable v_i follow a Poisson distribution, for a time period of length T ($T \gg w$), the expected interval length generated by v_i is

$$L = \begin{cases} T(1 - e^{-\lambda_i \cdot w}), & v_i \text{ is the head or tail variable,} \\ T(1 - e^{-2\lambda_i \cdot w}), & \text{otherwise,} \end{cases} \quad (2)$$

where $\lambda_i = \frac{N \cdot sel_i}{T}$ represents the event arrival rate corresponding to v_i , and sel_i is the overall selectivity of v_i .

Proof. We first derive the expected covered length of the replay interval set when v_i is the head variable. Clearly, for any time point $t \in [0, w)$, if the replay interval set RIS covers t , then there must exist an event e_j within the time interval $[0, t]$ that satisfies the constraint conditions of v_i . Similarly, for any time point $t \in [w, T]$, if it can be covered by RIS , then there

TABLE III: Details of the real-world and synthetic datasets.

Dataset	#events	Column
JOB [43]	2M	type, jobId, username, schedulingClass, jobName, timestamp
NASDAQ [15]	8.7M	ticker, open, high, low, close, vol, date
CRIMES [44]	7.7M	primaryType, id, beat, district, lat, lon, date
SD10M	10M	type, a1, a2, a3, a4, timestamp
SD100M	100M	type, a1, a2, a3, a4, timestamp
SD1G	1G	type, a1, a2, a3, a4, timestamp

must exist an event e_j within the time interval $[t - w, t]$ that satisfies the constraint conditions of v_i . Then, the probability that the time point t is covered is

$$P(t \text{ is covered}) = \begin{cases} 1 - e^{-\lambda_i t}, & t < w, \\ 1 - e^{-\lambda_i w}, & t \geq w. \end{cases}$$

Since the expected covered length L_i can be obtained by integrating the coverage probability over the entire time interval $[0, T]$, then the expected interval length generated by v_i is

$$L = \int_0^T P(t \text{ is covered}) dt = \int_0^w (1 - e^{-\lambda_i t}) dt + \int_w^T (1 - e^{-\lambda_i w}) dt.$$

When $T \gg \tau$, we have

$$L \approx \int_0^T (1 - e^{-\lambda_i \tau}) dt = T(1 - e^{-\lambda_i w}).$$

Due to the similarity of the derivation process, we omit the derivation of the other two cases. $\square$

Based on Theorem 2, the first processed variable is selected according to the minimum interval length to construct the initial RIS. The remaining variables are then prioritized by selectivity, with lower selectivity (*i.e.*, those matching fewer events) variables processed first. This ordering more effectively prunes the current candidate replay intervals in *RIS*, thereby reducing the number of Index Blocks that subsequent variables need to access.

V. EXPERIMENTAL EVALUATION

In this section, we first describe our experimental settings. Then, we assess the performance impact of various parameter configurations and optimizations. Lastly, we evaluate the performance of ACER and ACER-E against prior SOTA index-based and fullscan-based methods on six real-world and synthetic datasets. Our source code is publicly available [42].

A. Experimental Settings

Environment. All experiments are conducted on a Windows 11 PC with an Intel(R) Core(TM) i9-10900K CPU at 3.70 GHz. This machine is equipped with 16 GB memory at 3200 MHz (the maximum memory allocation pool of JVM is set to 4GB), and an ST2000DM008-2FR102 HDD. We implement all methods in Java, including a pattern matching algorithm based on NFA [17], which supports all temporal operators.

Datasets. Table III details each dataset used in our experiments. Attributes underlined are those involved in queries, and we build indexes for them. The three real-world datasets, *i.e.*, JOB, NASDAQ, and CRIMES, are sourced from prior works [20], [30], [32], which record job lifecycles, stock trading information, and crime reports, respectively. Their space overhead for a single event is 112, 48, and 40 bytes.

Besides, to achieve desired selectivity and for scalability testing, we generated three synthetic datasets: SD10M,

TABLE IV: Queried event patterns on synthetic datasets.

P1	SEQ(A a, B b, C c)
P2	SEQ(A a, B b, C c, D d, E e)
P3	SEQ(SEQ(A a, AND(B b, C c)), D d)
P4	AND(SEQ(A a, B b), SEQ(C c, D d))
P5	SEQ(AND(A a, B b), AND(C c, D d))

SD100M, and SD1G, with 10M, 100M, and 1G events, respectively (where 1K=1024, 1M=1K×1K, and 1G=1K×1M). For the three synthetic datasets, the event type obeys a Zipf distribution (skew=1.2, n=20) [45], attributes a1 and a2 are uniformly distributed over $\mathcal{U}[1, 1000]$, a3 and a4 follow a Gaussian distribution $\mathcal{N}(5000, 1000^2)$. The event arrival rate is one event per second, and the space overhead per event across these synthetic datasets is 36 bytes.

Baselines. We choose three baselines: (1) FullScan [23] scans all events from disk and pre-filters irrelevant events based on the window condition and IPCs before sending them to the evaluation engine; (2) NaiveIndex [20] constructs the tree-like indexes for the event type and attributes, and queries all type conditions and IPCs via the built indexes to obtain events; and (3) IntervalScan [20] additionally constructs an index for the timestamp column compared with NaiveIndex. It combines the window condition and partial IPCs to obtain relevant events (the detailed query plan is presented in §I). We use the open-source B+Tree [46] as the tree-like index structure for the baselines and set its order (*i.e.*, the maximum number of children that a node can have) to 128 since the range query performance is optimal under this setting. Lastly, we set the page size in all methods to 8 KB.

Queries. On JOB dataset, we identify patterns that jobs (with the same scheduling class) are submitted, scheduled, and evicted/finished/killed within a short period (50ms-1s) [30]. On the NASDAQ dataset, we search for price anomalies of two highly correlated stocks, where the opening price of one stock rises, and the opening price of the other stock falls within 15 minutes. On CRIMES dataset, we search a sequence of ROBBERY, BATTERY and MOTOR VEHICLE THEFT within 30 minutes, and the places where these events occurred are relatively near [20], [23]. Notably, we generate 500 queries for each real-world dataset. These queries choose *skip-till-next-match* as the selection strategy.

On the synthetic datasets, we evaluate five event patterns (each pattern generates 100 queries) listed in Table IV, where capital letters (A-E) represent an event type $type_i$. The event type of each variable is randomly selected and without duplication. Besides, we randomly set two IPCs for each variable (*the selectivity of each IPC is set to 10% by default*) and randomly add 1-3 DPCs for each query. For all queries on the three synthetic datasets, we set the query window to 1000s and set the selection strategy to *skip-till-any-match*.

B. Parameter Setting and Ablation Experiment

1) *Parameter Setting:* Fig. 9 and Fig. 10 show the changes in average query latency and index size under varying buffer pool capacities, respectively. Notably, at a capacity of 64K, the index size reaches a minimum value, and both ACER and ACER-E deliver optimal or near-optimal query performance. Thus, 64K is chosen as the default buffer pool capacity.

2) *Ablation Experiment:* Table V shows the ablation experiments on the three optimizations proposed in §IV (abbreviated

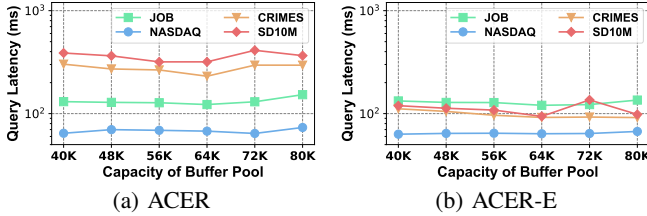


Fig. 9: Average query latency versus Buffer Pool's capacity.

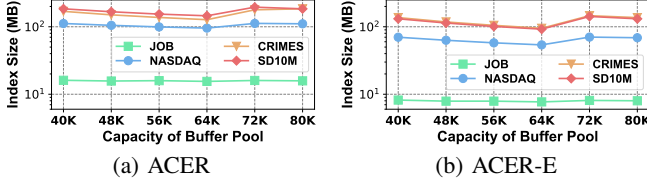


Fig. 10: Index size versus Buffer Pool's capacity.

as O1, O2, and O3). Enabling O1 can reduce the query latency by 33.6%, 10.5%, 63.2%, and 66.1%, as it reduces disk IO by loading index block content on demand. However, enabling O2 alone has a negative impact on query performance, as Delta-Zigzag-Simple8B algorithm takes a long time to decompress a list with 64K values. Fortunately, when both O1 and O2 are enabled, high-ratio block compression does not significantly affect query performance; instead, it reduces the index size (as shown in Fig. 10), making this optimization beneficial. Enabling O3 can reduce the query latency by 21.9%, 7.5%, 0.5%, and 3.3%, as it accesses fewer index blocks.

C. Results for Real-world Datasets

1) *Speedup Performance Comparison*: We first evaluate the latency of each query for four methods. Then, we calculate the query speedup of index-based methods compared with FullScan, and show the results in Fig. 11. Clearly, ACER always outperforms FullScan, achieving median speedups of 9.6 $\times$, 33.1 $\times$, and 5.1 $\times$ on the JOB, NASDAQ, and CRIMES datasets, respectively. ACER-E achieves median speedups of 11.7 $\times$, 36.6 $\times$, and 13.4 $\times$ since it optimizes the read/write operations of the Index Block and variable query order. In contrast, due to high index access and additional sort overhead, IntervalScan only achieves median speedups of 1.1 $\times$, 2.7 $\times$, and 0.5 $\times$ on the same datasets.

Both ACER and ACER-E achieve a high speedup for two main reasons: (1) their index access overhead is relatively low and can output an ordered result without sorting; and (2) they can filter out events that cannot participate in matching through the window-wise filtering algorithm. Compared with FullScan, IntervalScan can reduce query latency on JOB and NASDAQ datasets since the queries have a short query window or low predicate selectivity. Once the query has high predicate selectivity or a long query window (e.g., on Crimes dataset), IntervalScan struggles to accelerate CER. NaiveIndex needs to query all IPCs and type conditions using B+Tree, which incurs significant index access overhead. As a result, it commonly has higher query latency than FullScan.

2) *Insertion Throughput Comparison*: Fig. 12 shows the insertion throughput of varying methods on the three real-world datasets. Since FullScan does not build indexes on event attributes, it achieves the highest insertion throughput. Compared with tree-based indexing methods (i.e., NaiveIndex and IntervalScan), ACER and ACER-E improve insertion

TABLE V: Average query latency when enabling three varying optimization knobs (abbreviated as O1, O2, and O3).

	<i>JOB</i>	<i>NASDAQ</i>	<i>CRIMES</i>	<i>SD10M</i>
ACER	122.5ms	67.6ms	230.8ms	318.2ms
ACER w. O1	81.3ms	60.5ms	85.0ms	107.9ms
ACER w. O2	141.7ms	72.1ms	439.2ms	552.8ms
ACER w. O1+O2	82.6ms	58.8ms	87.0ms	100.8ms
ACER w. O3	95.7ms	62.5ms	229.6ms	307.6ms

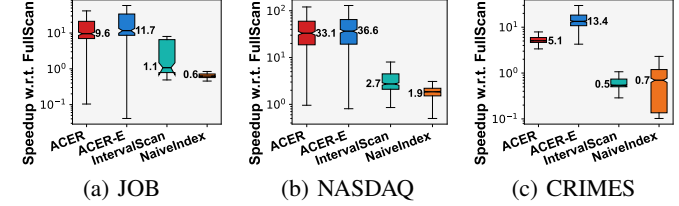


Fig. 11: Query speedup of index-based methods on the three real-world datasets.

throughput by about 54%, 160%, and 258% on the JOB, NASDAQ, and CRIMES datasets, respectively. The superior throughput of ACER and ACER-E stems from two factors: (1) their indexes are written in an append-only manner, avoiding random I/O, whereas tree-based indexes incur random writes; and (2) when inserting multiple attribute values for an event, they update a single index buffer before flushing, while tree-based indexes update multiple buffers, increasing flush frequency. ACER-E further compresses the timestamp and RID columns. While this compression introduces additional computational overhead, it reduces write I/O overhead. As a result, ACER-E achieves 35% and 6% higher insertion throughput than ACER on the JOB and NASDAQ datasets.

3) *Space Overhead Comparison*: Fig. 13 reports the disk space overhead across different methods, defined as the sum of the raw data size and the index size. Compared with FullScan, ACER and ACER-E only increase space overhead by around 4%, 14%, and 32% on the three real-world datasets. They maintain a small index footprint for two main reasons: (1) type information is stored only once per cluster; and (2) the Range Bitmap index, timestamp list, and RID list within each Index Block store compressed rather than raw values, significantly reducing storage overhead. In contrast, IntervalScan increases space consumption by approximately 3.1 $\times$, 5.1 $\times$, and 8.6 $\times$ on the same datasets. Its large index size stems from three factors: (1) it constructs additional indexes for the type and timestamp columns compared with ACER; (2) redundant information is stored across multiple B+Tree indexes; and (3) many B+Tree nodes are non-full. As a result, IntervalScan incurs substantial I/O overhead when accessing indexes.

Based on the results for query latency, insertion throughput, and space overhead, our proposed indexes achieve both low maintenance cost and high query efficiency. These advantages stem from three aspects: the Range Bitmap, which supports fast multi-attribute range queries via bitwise operations; the window-wise filtering, which aggressively prunes irrelevant events based on window conditions; and the efficient index organization that avoids random I/O during writes and incurs low storage overhead.

4) *Query Latency Comparison in Real Systems*: We conduct end-to-end latency in three real systems: Flink [14], Apache Pinot [47] (which provides Range Bitmaps for index-

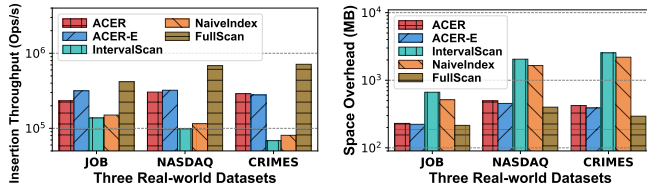


Fig. 12: Comparison of insertion throughput. Fig. 13: Comparison of total space overhead.

TABLE VI: Comparison of the end-to-end latency in real systems, where the six queries $Q_1 \sim Q_6$ are available [42].

Dataset & Query	NASDAQ		JOB		CRIMES	
	Q_1	Q_2	Q_3	Q_4	Q_5	Q_6
Flink w. ACER	1.24s	1.66s	1.06s	1.14s	1.06s	1.09s
Flink w. ACER-E	1.15s	1.55s	1.04s	1.09s	0.89s	0.92s
Flink w. IntervalScan	1.48s	3.41s	2.33s	3.46s	1.70s	2.78s
Flink w. FullScan	2.66s	3.11s	1.63s	1.75s	1.87s	1.89s
Flink w. Pinot	1.89s	2.19s	5.07s	6.39s	1.36s	1.37s
DBMS w. Rewriting	316.3s	755.4s	1.35s	1.46s	2.97s	8.04s

ing attribute values), and a commercial database management system supporting CER, referred to as DBMS. To minimize additional overhead, we feed the events obtained from ACER, the baseline methods, and Pinot directly into Flink's input stream. Additionally, we build bitmap-based indexes for queried attributes in Pinot. Remarkably, we observe that when using Flink as the evaluation engine, matching becomes the query bottleneck since Flink has a cold start delay of about 0.8s. Table VI reports the average query latency across these systems, with each query executed 10 times.

Compared with FullScan, ACER reduces the query latency by 34% to 53%, as ACER has a lower cost of loading events from disk to memory than FullScan, and the number of events it sends to Flink is also slightly less than that of FullScan. Since ACER-E enhances index block read/write operations and variable processing order, its query latency is lower than that of ACER. Compared with Pinot, ACER reduces the query latency by 20%~82%. The reason is that ACER can filter out more irrelevant events and can avoid unnecessary disk I/O via window-wise filtering. For example, on JOB dataset, ACER loads only 2% or fewer of the events that Pinot needs to access from disk under a short window size. As for IntervalScan, it may send more events to Flink than FullScan, even if it has lower latency when loading events. Consequently, its overall query latency may be higher than FullScan. Lastly, the average query latency of the DBMS is more than 2.8 times higher than that of Flink w. FullScan, due to the lack of early termination based on window conditions.

5) *Query Latency Comparison for Special Operators:* We continue to evaluate the average latency of three queries $Q_7 \sim Q_9$ (they have an additional variable bound by a special operator compared with Q_5) that contain special operators on CRIMES dataset (each query runs 10 times) and show the results in Table VII. Since the window-wise filtering still works for these queries, ACER reduces FullScan's query latency by over 29%, and ACER-E reduces FullScan's query latency by over 48%. As for IntervalScan, it has higher query latency than FullScan for queries $Q_7 \sim Q_9$, as it sends more events to Flink. As a result, IntervalScan's latency exceeds that of FullScan by over 1.3 $\times$.

TABLE VII: Comparison of the end-to-end latency in Flink, here $Q_7 \sim Q_9$ on CRIMES dataset are available [42].

	$Q_7(w. KP)$	$Q_8(w. KS)$	$Q_9(w. NEG)$
Flink w. ACER	1.15s	1.32s	1.33s
Flink w. ACER-E	0.97s	0.99s	0.99s
Flink w. IntervalScan	2.87s	2.91s	2.66s
Flink w. FullScan	1.90s	1.94s	1.90s

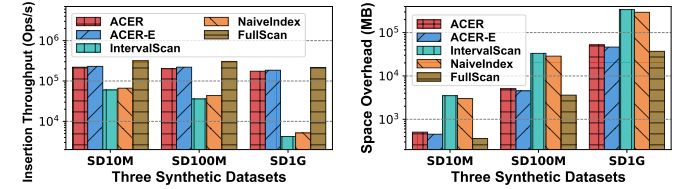


Fig. 14: Comparison of insertion throughput. Fig. 15: Comparison of total space overhead.

D. Results for Synthetic Datasets

1) *Average Query Latency Comparison:* Table VIII reports the average query latency on the three synthetic datasets. Compared with other baselines, ACER reduces query latency by up to 4 $\times$, while ACER-E achieves over an order of magnitude latency reduction on SD10M and SD100M datasets. On SD1G dataset, all methods experience a significant latency increase. This is primarily because the total dataset size (36 GB) substantially exceeds the JVM heap capacity (4 GB), which frequently triggers page faults and costly memory-mapped file (mmap) swapping when retrieving events through RID list. Despite the increased I/O pressure, ACER and ACER-E still maintain their superiority, reducing query latency by 54%~86% and 72%~93%, respectively. Compared with FullScan, neither IntervalScan nor NaiveIndex can improve query performance. The reason is that the selectivity of each IPC is relatively high (*i.e.*, 10%), using B+Tree to retrieve events cannot bring any performance benefit [36]. Although IntervalScan only queries partial conditions with a low selectivity (*i.e.*, type conditions), it still loads numerous irrelevant events from the disk under a very long query window. Consequently, the latency of IntervalScan and NaiveIndex is prohibitively expensive, and their latency surprisingly exceeds 20 minutes on SD1G dataset.

2) *Insertion Throughput Comparison:* Fig. 14 reports the insertion throughput of varying methods on the three synthetic datasets. As the number of inserted events increases, the insertion throughput of ACER, ACER-E, and FullScan decreases slightly due to longer disk rotation and seek delays. In contrast, IntervalScan and NaiveIndex exhibit a sharp throughput degradation. On SD1G dataset, their throughput is less than 5% of that achieved by our proposed indexes. This low throughput can be attributed to three factors: (1) as the data volume grows, the height of the B+Tree increases, resulting in more I/O operations per insertion; (2) B+Tree insertions incur frequent random disk writes; and (3) maintaining multiple B+Tree indexes amplifies random I/O, since each index is stored in a separate file.

3) *Space Overhead Comparison:* Fig. 15 shows the total disk space overhead of the varying methods. The total space overhead of ACER and ACER-E is about 1.4 $\times$ and 1.25 $\times$ that of FullScan, respectively. In contrast, IntervalScan and NaiveIndex incur substantially higher overhead, exceeding

TABLE VIII: Comparison of the average query latency on the three synthetic datasets (TO represents the latency > 20 minutes).

Dataset & Pattern	SD10M					SD100M					SD1G				
	P1	P2	P3	P4	P5	P1	P2	P3	P4	P5	P1	P2	P3	P4	P5
ACER	0.29s	0.31s	0.31s	0.35s	0.35s	2.92s	3.28s	3.24s	3.45s	3.39s	242.0s	73.8s	108.6s	144.8s	151.4s
ACER-E	0.10s	0.08s	0.09s	0.11s	0.11s	0.99s	0.84s	0.92s	1.09s	1.09s	147.1s	35.3s	63.4s	103.4s	99.4s
IntervalScan	4.00s	4.10s	3.99s	3.96s	3.96s	75.1s	57.9s	54.5s	64.4s	65.8s	TO	TO	TO	TO	TO
NaiveIndex	4.02s	6.60s	5.38s	5.39s	5.41s	72.0s	103.9s	89.8s	89.6s	89.8s	TO	TO	TO	TO	TO
FullScan	1.33s	1.49s	1.34s	1.35s	1.35s	13.63s	15.21s	13.56s	13.71s	13.74s	529.1s	539.8s	517.5s	510.8s	501.0s

TABLE IX: Comparison of the average operation latency.

Method	Operation	Update/deletion Ratio					
		0%	10%	20%	30%	40%	50%
ACER	IDU (μ s)	4.6	6.5	6.3	6.2	6.2	6.1
	Query (ms)	318	349	366	399	451	455
ACER-E	IDU (μ s)	4.3	6.4	6.1	6.3	6.4	5.6
	Query (ms)	105	111	117	134	135	137

9.1 $\times$ and 7.9 $\times$ that of FullScan, respectively. These results demonstrate that our proposed indexes achieve low storage overhead via attribute compression and the elimination of redundant information, which reduces index maintenance cost.

4) *Impact of Deletion/Update on Query Latency*: We first define the ratio of deletion/update as the number of deletion and update operations divided by the number of insertion operations. Table IX shows IDU (insertion, deletion, and update) and query latency of ACER and ACER-E at varying update/deletion ratios, where updates are fixed at a predefined value, the number of deletion and update operations is equal, and the number of insertion operations is 10M. Logical deletion and update operations cause out-of-order events and additional sorting, resulting in higher IDU latency for ACER and ACER-E compared with insert-only workloads. Regarding query performance, logical updates and deletions increase the number of indexed attribute values and Index Blocks, which in turn leads to higher average query latency.

5) *Impact of Events Arrive Out-of-order on Query Latency*: We simulate the arrival of out-of-order events by randomly batch-shuffling ordered event streams of varying sizes and generate 10M events. The results are shown in Fig. 16. Since events are appended to the Page directly, when events arrive out-of-order, loading them from the disk involves random access. Fortunately, ACER and ACER-E can filter many events via window-wise filtering, thereby alleviating the significant increase in query latency that random access would cause.

6) *Sensitivity Evaluation of Pattern Length, Selectivity, and Window Size*: We construct the pattern $SEQ(type_{s_1}v_1, \dots, type_{s_\xi}v_\xi)$ that has varying pattern length from 3 to 7. We generate 100 queries with the same predicate and event type condition setting as described in §V-A for each pattern length (note that we only change the patterns when conducting sensitivity experiments of pattern length, others still use previous patterns). All the following experiments are conducted on SD10M dataset.

Fig. 17 shows the average query latency of all methods in varying pattern lengths. As the pattern length increases, the overhead of reading Index Blocks grows, but the number of matches decreases, reducing the number of events that need to be accessed by ACER and ACER-E. Consequently, their overall latency remains relatively stable. In comparison, NaiveIndex also evaluates all type conditions and IPCs, but its expensive index lookups incur substantial overhead, leading to

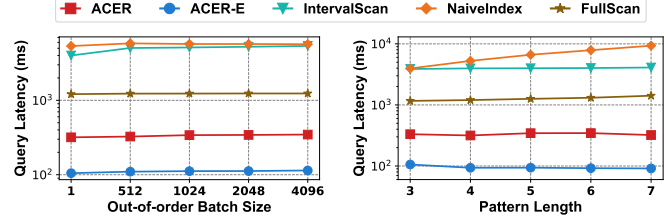


Fig. 16: Average query latency versus batch size. Fig. 17: Average query latency versus pattern length.

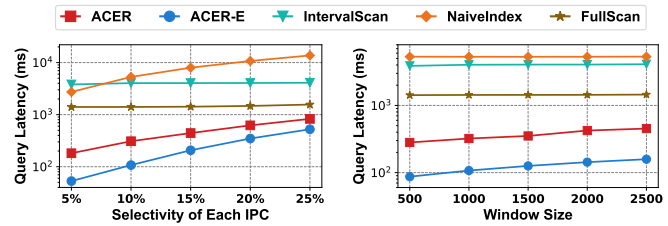


Fig. 18: Average query latency versus predicate selectivity. Fig. 19: Average query latency versus window size.

higher query latency for longer patterns. IntervalScan selects only a subset of constraint conditions with minimum selectivity to generate replay intervals, thereby avoiding additional overhead as the pattern length increases. FullScan maintains stable performance because it always scans the entire dataset from disk, where I/O cost dominates the total query time.

Fig. 18 shows the average query latency of all methods for 500 queries under varying selectivity of IPC, ranging from 5% to 25%. Since ACER, ACER-E, and NaiveIndex query all IPCs, higher selectivity leads to higher query latency for them. Fortunately, ACER and ACER-E have a low index query overhead, thereby they can still reduce query latency by over 45% under 25% selectivity of IPC, compared with FullScan. Since FullScan loads all events from disk to memory and the scan cost dominates the overall query latency, the increased selectivity does not cause a significant increase in overall query latency. IntervalScan does not prioritize predicates with high selectivity for filtering. As a result, its query latency remains relatively stable as IPC selectivity increases.

Fig. 19 shows the average latency over 500 queries for each method across varying window sizes. As the window size grows, the average query latency of ACER and ACER-E rises. The reason is that the window condition becomes less selective, reducing pruning of Index Blocks and events, and thereby increasing access cost. Regarding IntervalScan, as the window size increases, IntervalScan tends to select more query conditions to reduce disk I/O overhead. As a result, larger windows do not significantly impact IntervalScan's query latency. For NaiveIndex and FullScan, the trend of increased latency is not apparent, as the cost of expensive disk access and filtering dominates the query latency.

VI. RELATED WORK

Complex event recognition (CER) in event stores. CER aims to identify specific patterns from historical data. Typically, event stores collect events from various sources and respond to query patterns after retrieving the events from the stores. Thus, CER is *human-active database-passive* [48]. Some implementations of event stores for CER include Cayuga [16], ChronicleDB [8], and CORE [19]. Yet, these systems often suffer from high query latency due to costly disk I/O overhead and the filtering of the evaluation engine. To reduce high query latency, (1) Zhu *et al.* [23] introduced a bucketized pre-filtering approach that effectively reduces matching costs, but it cannot alleviate the costly I/O overhead; (2) Korber *et al.* [20] proposed an index filtering approach that reduces I/O access in events. However, this method incurs extra and expensive index accessing and sorting overhead. As a result, it still struggles with high latency, particularly for queries with high predicate selectivity or long query window. In contrast, our ACER structure efficiently alleviates costly disk I/O overhead and automatically produces ordered results without sorting, thereby reducing overall query latency.

Complex event processing (CEP) in data-stream systems. CEP focuses on real-time monitoring and processing of pre-defined patterns from streaming data. Specifically, the user first specifies the predetermined detection patterns to data-stream systems, and then the systems consume events to extract matches and trigger planned actions. Thus, the interaction in CEP is *human-passive database-active* [48]. A practical data-stream system for CEP is Flink [14]. Commonly, the data-stream systems use NFA as the evaluation engine [17], [18], [21], [49]. However, NFA often encounters low processing throughput. To address this issue, various techniques have been proposed, including parallel-based techniques [45], [50]–[54], join-based techniques [28], [55]–[58], sub-pattern sharing techniques [31], [59]–[61], distributed-based [26], [27], [33], [62], and filter-based techniques [10], [24], [32] (a comprehensive survey can be found in [1]). Notably, CEP runs in an online scenario where every incoming event is examined and stored in main memory [20]. In contrast, we aim to reduce the I/O overhead when loading the events to the main memory. Thus, the optimization techniques in CEP are typically orthogonal to our solution.

VII. CONCLUSION

This paper proposes Range Bitmap-based indexes, namely ACER and ACER-E, for accelerating CER. The main ideas are reducing the index block size to alleviate I/O overhead, constructing Range Bitmaps for query attributes to produce ordered results without sorting, and utilizing a window-wise filtering algorithm to avoid unnecessary disk access. The experimental results show that our indexes reduce the query latency by up to one order of magnitude on six real-world and synthetic datasets over SOTA techniques.

REFERENCES

- [1] N. Giatrakos *et al.*, “Complex Event Recognition in the Big Data Era: A Survey,” *The VLDB Journal*, vol. 29, no. 1, pp. 313–352, 2020.
- [2] H. Zhang *et al.*, “On Complexity and Optimization of Expensive Queries in Complex Event Processing,” in *SIGMOD*, 2014, pp. 217–228.

- [3] S. Kleest-Meißner *et al.*, “Discovering Event Queries from Traces: Laying Foundations for Subsequence-Queries with Wildcards and Gap-Size Constraints,” in *ICDT*, 2022, pp. 18:1–18:21.
- [4] S. Song *et al.*, “Why Not Match: On Explanations of Event Pattern Queries,” in *SIGMOD*, 2021, pp. 1705–1717.
- [5] S. Dharmapurikar *et al.*, “Fast and Scalable Pattern Matching for Network Intrusion Detection Systems,” *IEEE Journal on Selected Areas in Commun.*, vol. 24, no. 10, pp. 1781–1792, 2006.
- [6] L. Wu *et al.*, “Building Intrusion Pattern Miner for Snort Network Intrusion Detection System,” *Journal of Systems and Software*, vol. 80, no. 10, pp. 1699–1715, 2007.
- [7] B. Dong *et al.*, “Efficient Discovery of Abnormal Event Sequences in Enterprise Security Systems,” in *CIKM*, 2017, pp. 707–715.
- [8] M. Seidemann *et al.*, “ChronicleDB: A High-Performance Event Store,” *ACM Trans. Database Syst.*, vol. 44, no. 4, pp. 13:1–13:45, 2019.
- [9] K. Golmohammadi *et al.*, “Data Mining Applications for Fraud Detection in Securities Market,” in *EISIC*, 2012, pp. 107–114.
- [10] S. Huang *et al.*, “T-REX: Optimizing Pattern Search on Time Series,” *Proc. ACM Manag. Data*, vol. 1, no. 2, pp. 130:1–130:26, 2023.
- [11] ISO, “ISO/IEC 19075-5:2021 Information technology — Guidance for the use of database language SQL — Part 5: Row pattern recognition,” 2021, <https://www.iso.org/standard/78936.html>.
- [12] Oracle, “SQL for Pattern Matching,” 2017, <https://docs.oracle.com/database/121/DWHSG/pattern.htm>.
- [13] Snowflake, “Identifying Sequences of Rows That Match a Pattern,” 2020, <https://docs.snowflake.com/en/user-guide/match-recognize-introduction>.
- [14] “Apache Flink,” <https://flink.apache.org/>.
- [15] “Eoddata,” <https://www.eoddata.com>.
- [16] A. J. Demers *et al.*, “Cayuga: A general purpose event monitoring system,” in *CIDR*, 2007, pp. 412–422.
- [17] J. Agrawal *et al.*, “Efficient Pattern Matching Over Event Streams,” in *SIGMOD*, 2008, pp. 147–160.
- [18] D. Gyllstrom *et al.*, “SASE: Complex Event Processing over Streams (Demo),” in *CIDR*, 2007, pp. 407–411.
- [19] M. Bucchi *et al.*, “CORE: a Complex event Recognition Engine,” *PVLDB*, vol. 15, no. 9, pp. 1951–1964, 2022.
- [20] M. Körber *et al.*, “Index-Accelerated Pattern Matching in Event Stores,” in *SIGMOD*, 2021, pp. 1023–1036.
- [21] E. Alevizos *et al.*, “Complex Event Recognition with Symbolic Register Transducers,” *PVLDB*, vol. 17, no. 11, pp. 3165–3177, 2024.
- [22] J. García *et al.*, “Complex Event Recognition under Time Constraints: Towards a Formal Framework For Efficient Query Evaluation,” *Proc. ACM Manag. Data (PODS)*, vol. 3, no. 2, pp. 94:1–94:17, 2025.
- [23] E. Zhu *et al.*, “High-Performance Row Pattern Recognition Using Joins,” *PVLDB*, vol. 16, no. 5, pp. 1181–1195, 2023.
- [24] B. Cadonna *et al.*, “Efficient Event Pattern Matching with Match Windows,” in *KDD*, 2012, pp. 471–479.
- [25] A. Jaimes *et al.*, “Real-time Event Detection for Emergency Response Tutorial,” in *KDD*, 2021, pp. 4042–4043.
- [26] S. Akili *et al.*, “MuSE Graphs for Flexible Distribution of Event Stream Processing in Networks,” in *SIGMOD*, 2021, pp. 10–22.
- [27] Samira Akili *et al.*, “INEv: In-Network Evaluation for Event Stream Processing,” *Proc. ACM Manag. Data*, vol. 1, no. 1, pp. 101:1–101:26, 2023.
- [28] Y. Mei *et al.*, “ZStream: A Cost-based Query Processor for Adaptively Detecting Composite Events,” in *SIGMOD*, 2009, pp. 193–206.
- [29] A. Grez *et al.*, “A Formal Framework for Complex Event Recognition,” *ACM Trans. Database Syst.*, vol. 46, no. 4, pp. 16:1–16:49, 2021.
- [30] B. Zhao *et al.*, “EIRES: Efficient Integration of Remote Data in Event Stream Processing,” in *SIGMOD*, 2021, pp. 2128–2141.
- [31] K. Chapnik *et al.*, “DARLING: Data-Aware Load Shedding in Complex Event Processing Systems,” *PVLDB*, vol. 15, no. 3, pp. 541–554, 2021.
- [32] A. Amir *et al.*, “DLACEP: A Deep-Learning Based Framework for Approximate Complex Event Processing,” in *SIGMOD*, 2022, pp. 340–354.
- [33] S. Pirtzel *et al.*, “Predicate-based Push-pull Communication for Distributed CEP,” in *DEBS*, 2022, pp. 31–42.
- [34] P. E. O’Neil *et al.*, “Improved Query Performance with Variant Indexes,” in *SIGMOD*, 1997, pp. 38–49.
- [35] C. Y. Chan *et al.*, “Bitmap Index Design and Evaluation,” in *SIGMOD*, 1998, pp. 355–366.
- [36] B. Hentschel *et al.*, “Column Sketches: A Scan Accelerator for Rapid and Robust Predicate Evaluation,” in *SIGMOD*, 2018, pp. 857–872.
- [37] L. Li *et al.*, “BinDex: A Two-Layered Index for Fast and Robust Scans,” in *SIGMOD*, 2020, pp. 909–923.

- [38] Y. Chen *et al.*, "Cabin: A Compressed Adaptive Binned Scan Index," *Proc. ACM Manag. Data*, vol. 2, no. 1, pp. 1–26, 2024.
- [39] R. Startin, "RangeBitmap - How Range Indexes Work in Apache Pinot," <https://richardstartin.github.io/posts/range-bitmap-index>.
- [40] Protocol Buffers - Google's Data Interchange Format. [Online]. Available: <https://github.com/protocolbuffers/protobuf>
- [41] V. N. Anh *et al.*, "Index Compression using 64-bit Words," *Software Prac. Experience*, vol. 40, no. 2, pp. 131–147, 2010.
- [42] ACER authors, "ACER source code", 2024. [Online]. Available: <https://github.com/Josehokec/ACER4CER>
- [43] J. Wilkes, "Yet More Google Compute Cluster Trace Data," Google research blog, Apr. 2020, posted at <https://ai.googleblog.com/2020/04/yet-more-google-compute-cluster-trace.html>.
- [44] "Crimes - 2001 to present," <https://data.cityofchicago.org/stories/s/5cd6-ry5g>.
- [45] S. Akili *et al.*, "DecoPa: Query Decomposition for Parallel Complex Event Processing," *Proc. ACM Manag. Data*, vol. 2, no. 3, p. 132, 2024.
- [46] D. Moten, "bplustree," 2019, <https://github.com/davidmoten/bplustree>.
- [47] "Apache Pinot," 2019, <https://pinot.apache.org/>.
- [48] G. Cugola *et al.*, "Processing Flows of Information: From Data Stream to Complex Event Processing," *ACM Comput. Surveys*, vol. 44, no. 3, pp. 1–62, 2012.
- [49] Y. Diao *et al.*, "Sase+: An Agile Language for Kleene Closure over Event Streams," University of Massachusetts, Tech. Rep., 2007.
- [50] M. Yankovitch *et al.*, "HYPERSONIC: A Hybrid Parallelization Approach for Scalable Complex Event Processing," in *SIGMOD*, 2022, pp. 1093–1107.
- [51] I. Kolchinsky *et al.*, "Real-time Multi-Pattern Detection over Event Streams," in *SIGMOD*, 2019, pp. 589–606.
- [52] S. Zhang *et al.*, "Multi-Query Optimization for Complex Event Processing in SAP ESP," in *ICDE*, 2017, pp. 1213–1224.
- [53] T. Ohara *et al.*, "Fast SQL/Row Pattern Recognition Query Processing Using Parallel Primitives on GPUs," in *DEXA*, vol. 12923, 2021, pp. 22–34.
- [54] M. Kuhr *et al.*, "iGPU-Accelerated Pattern Matching on Event Streams," in *SIGMOD, DaMoN*, 2022, pp. 6:1–6:7.
- [55] I. Kolchinsky *et al.*, "Join Query Optimization Techniques for Complex Event Processing Applications," *PVLDB*, vol. 11, pp. 1332–1345, 2018.
- [56] Ilya Kolchinsky and others, "Efficient Adaptive Detection of Complex Event Patterns," *PVLDB*, vol. 11, pp. 1346–1359, 2018.
- [57] Ariane Ziehn and others, "Enabling Complex Event Processing in NebulaStream," in *EDBT*, 2025, pp. 1130–1133.
- [58] A. Ziehn *et al.*, "Bridging the Gap: Complex Event Processing on Stream Processing Systems," in *EDBT*, 2023, pp. 447–460.
- [59] L. Ma *et al.*, "Gloria: Graph-based Sharing Optimizer for Event Trend Aggregation," in *SIGMOD*, 2022, pp. 1122–1135.
- [60] O. Poppe *et al.*, "To Share, or not to Share Online Event Trend Aggregation Over Bursty Event Streams," in *SIGMOD*, 2021, pp. 1452–1464.
- [61] M. Ray *et al.*, "Scalable Pattern Sharing on Event Streams," in *SIGMOD*, 2016, pp. 495–510.
- [62] M. Akdere *et al.*, "Plan-based Complex Event Detection Across Distributed Sources," *PVLDB*, vol. 1, no. 1, pp. 66–77, 2008.



Shizhe Liu received the BS degree in software engineering from Jilin University, Jilin, China, in 2021. He is currently working toward the PhD degree in Nanjing University. His research interests include complex event processing and high-performance index structures.



Haipeng Dai (Senior Member, IEEE) received the Ph.D. degree from Nanjing University in 2014. He is an associate professor in Nanjing University. His research interests include data mining and mobile computing. He has published research papers in ACM SIGMOD, VLDB, IEEE ICDE, ACM WWW, ACM SIGMETRICS, IEEE INFOCOM, IEEE TKDE, IEEE TPDS, and IEEE TMC.



Shaoxu Song (Senior Member, IEEE) is an associate professor with the School of Software, Tsinghua University, Beijing, China. His research interests include data quality and complex event processing. He has published more than 100 papers in top conferences and journals such as ACM Transactions on Database Systems, IEEE Transactions on Knowledge and Data Engineering, VLDB Journal, SIGMOD, VLDB, ICDE, SIGKDD, *etc.*



Meng Li received the Ph.D. degree from Nanjing University in 2023. He is an assistant researcher in Nanjing University. His research interests are in the area of high-performance query processing in databases. He has published research papers in SIGMOD, VLDB, ICDE, SIGKDD, WWW, INFOCOM, IEEE TKDE, IEEE/ACM TON, and VLDB Journal.



Yuemeng Zhang received the BE degree in Internet of Things engineering from Chengdu University of Technology, Sichuan, China, in 2024. He is currently working toward the MS degree in Nanjing University. His research interests include edge computing and edge intelligence.



Hancheng Wang (Graduate Student Member, IEEE) received the BE degree from Jilin University in 2020. He is currently working toward the PhD degree in Nanjing University. His research interests include high-performance query processing and query optimization.



Du Zhang is currently pursuing the B.E. degree in information security at Nankai University, Tianjin, China. His research interests include edge intelligence and network security.



Rong Gu (Member, IEEE) received the Ph.D. degree from Nanjing University in 2016. He is a research associate professor in Nanjing University. His research interests include big data and cloud computing systems, efficient caching, and indexing. His research papers have been published in many conferences and journals, including IEEE TPDS, IEEE ICDE, IEEE IPDPS, IEEE ICPP, JSA, Parallel Computing, JPDC, and SPE.



Guihai Chen (Fellow, IEEE) received the Ph.D. degree from the University of Hong Kong in 1997. He is a professor and deputy chair of the Department of Computer Science, Nanjing University, China. He had been invited as a visiting professor by many foreign universities, including Kyushu Institute of Technology, University of Queensland, and Wayne State University, USA. His research interests include parallel computing, high-performance computing, and data engineering.